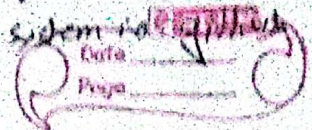


G.I

a) → How to init git in local system

b) → How to init git in remote system

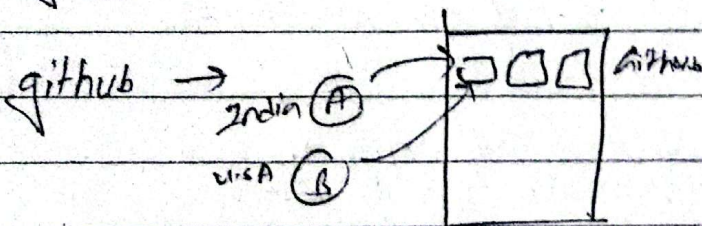
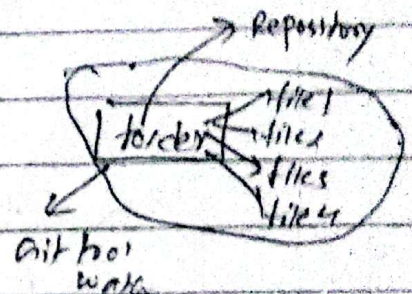
~~Git~~ → setting



1. What is Version Control?

- Make save points that saves your project.
- provides total development freedom.
- e.g. git, Subversion, perforce
- Git is a version control tool.
- Github is a service that hosts git repositories. (collaboration)

Repository → folder watched/tracked
is contained inside by a version control tool ie git
github.



git → track the changes or modification in the files.

After installing git

- go to cmd
- to check git version: `git --version`
- `git config --global user.name "sejan"` set username to "sejan" in git environment
- `git config --global user.email "Sujithap12345@gmail.com"`
- to finish setup in windows (git setup).

* To access hidden files in cmd, we use command `dir /a`

- a) To access all files i.e. hidden + unhidden → `dir /a`
 b) To access only hidden files → `dir /a:h` or `dir /ah`
 c) To access only unhidden files → `dir`

DATE _____
 PAGE _____

6. Creating a GIT Repository.

* Make sure to install git on desktop.

Step 1: create a folder and convert it to git repository.

a) open "cmd" and change directory to folder location
 i.e. `cd` ~~to~~ file location.

b) `git init` should be used to convert empty folder to empty repository i.e. `git init` command create a hidden folder named `.git` (by default hidden)

Note: • → period • • → period period

`cd .` → move to current directory

`cd ..` → move terminal to parent directory

`.git` → contains → meta data of git repository.
 (useful information used by ~~git~~ version control tool
 i.e. git)

07. How to Commit ~~to~~ in git. a) Newly created file:

* create a new file inside a ^{newly} created repository folder.

* ~~Working~~ File is untracked (because working directory).

Working directory → staging index (about to commit)
 or file is tracked

Command:

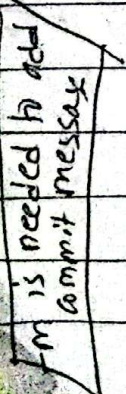
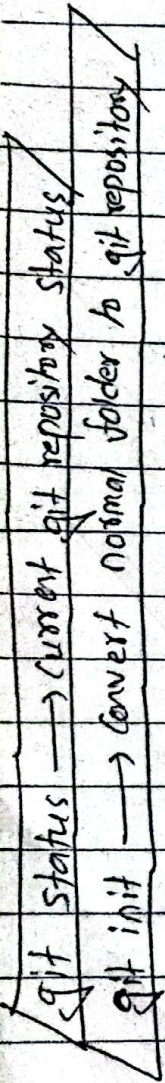
`git add filename`

(file is tracked)

* Staging index → Respi Repository (file is committed)

Command: `git commit -m "Commit msg"`

filename is optional.



to see modification / changes $\rightarrow$ $\left\{ \begin{array}{l} \text{git show} \rightarrow \text{to see changes of 1 commit} \\ \text{git log --p} \rightarrow \text{to see changes of entire commit} \end{array} \right.$

to see changes of **entire** commit

Date: _____
Page: _____

b) modified file

What when file is modified

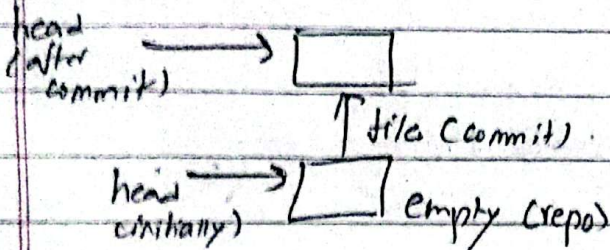
1) `git diff filename` $\rightarrow$ to see modification

2) `git add filename` $\rightarrow$ to add it to staging index

`git add .` $\rightarrow$ not recommended (put all ^{modify} files into staging index)
 $\rightarrow$ because files which we don't want to ~~not~~ modify are all put in staging index

3) `git commit filename -m "commit msg"`

* What is head $\rightarrow$ head always point to recent commit. ~~or~~ latest commit.
 just like `shch` points to `to.s`.



c) unwanted modified file (I ~~not~~ don't want to modify update)

step 1: `git status`

step 2: `git diff` (if changes content error/wrong msg then we want to discard it)

step 3: `git restore filename` (to discard changes)

Restore take place at head (latest working commit) $\rightarrow$ because it points (latest commit)

git diff → used to view modify files.

xx folder and git repository differs by .git folder

GROUP

Date _____
Page _____

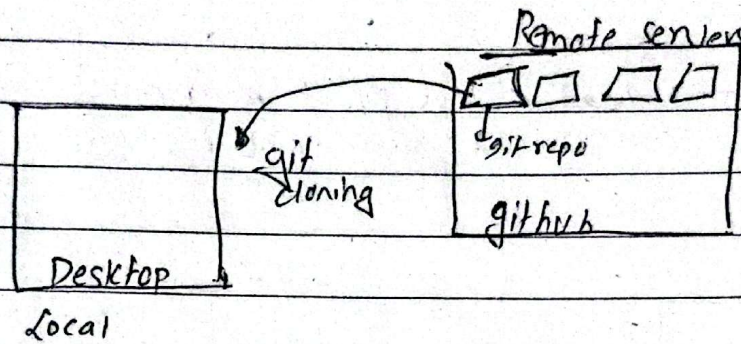
Command

6. Clone a git repository: • Git clone

Step 1: copy the url of repository from Code/Buttons which you want to clone from server/websites.

Step 2: `git clone [url] foldername (git repository)`

foldername is optional. i.e. foldername is written then files are stored in that foldername (means git repository)

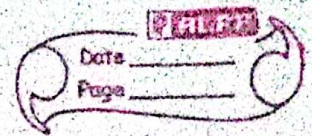


git cloning → means copying a repository from Remote/server to a local/desktop.

* To remove file from staging index: i.e. unstage

~~git rm --file-name~~ `git rm --cached file-name`

Every file have commit id → Which tells whether it is old commit or new commit.



7. Lifecycle of a git

Suppose: git folder (repository) (after modification)

i) file.js → file1'.js
ii) file2.js → file2'.js
iii) file3.js → file3'.js

* How to commit a change/modification i.e. finalize a change.

a) Working dir → staging index → git add command

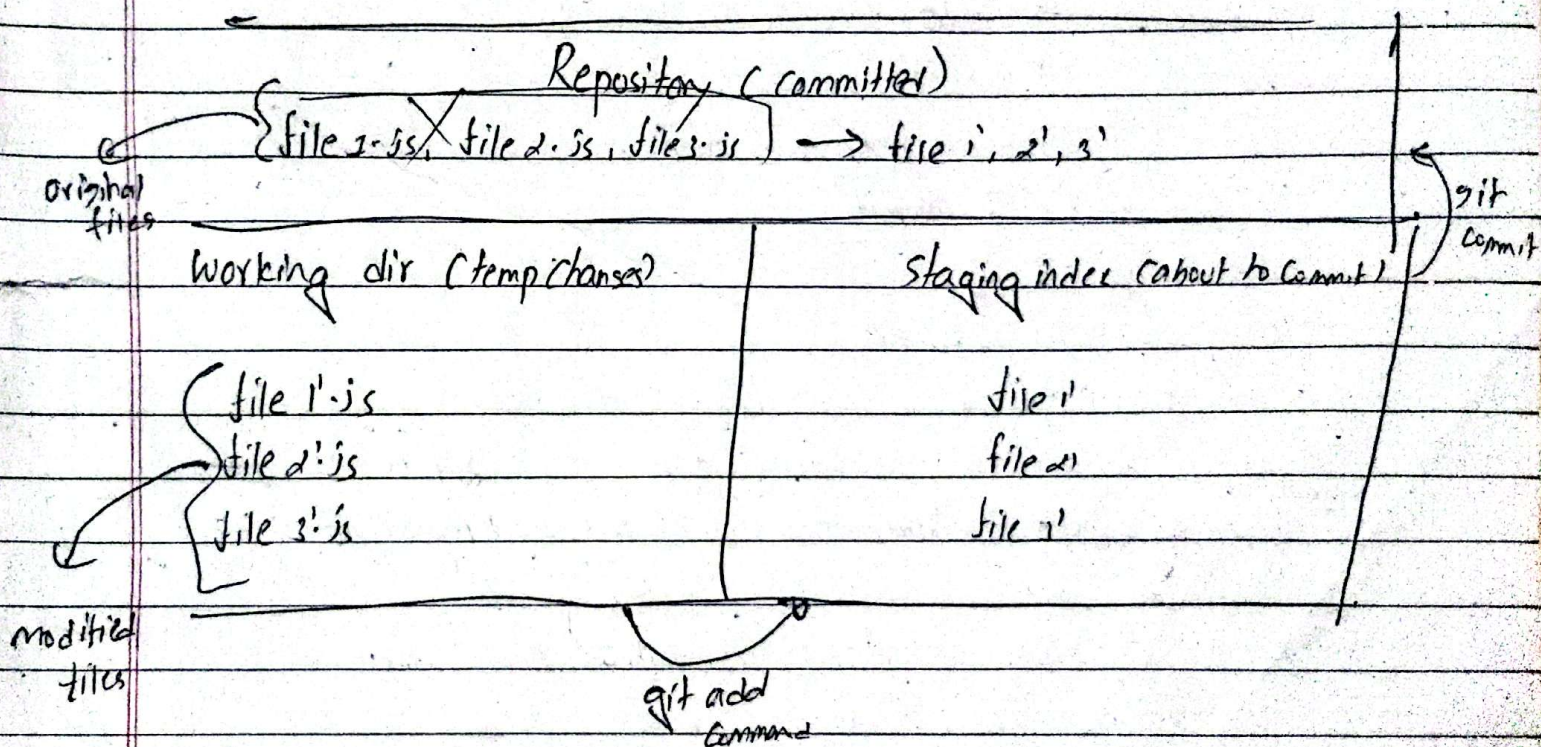
b) staging index → Repository → git commit command

Git repository → 3 areas

a) Working directory (temp. changes)

b) Staging index (about to commit)

c) Repository (committed)



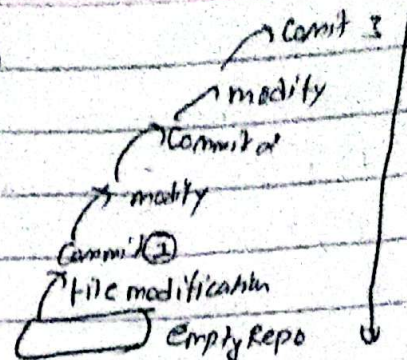
file1.js → old commit id → let say commit id → ABC

file2'.js → new commit id → let say ... → ABC2

Commit → means save point

08. Reviewing History in a Git Repo

- `git log`
- `git log -p`
- `git log --stat`
- `git show`



* To check ^{of view} commit in git (what are commit)
`git log` → top comment / latest comment to old commit display.
press 'q' to stop display displaying commit.

`commit` → commit id
→ Author
→ date
→ commit msg are display.

To see latest 3 commit

`git log -3`

To see latest 2 commit

`git log -2`

* `git log -p` → display commit as well as ^{info} modification i.e. ^{diff} changes.

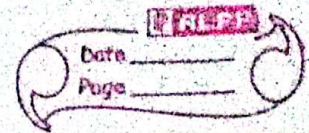
* `git log --oneline` → display commit id and commit message
↳ to list commit msg in one line.

* To see file name which have been modified, but don't show the context that have been modified.

`git log --stat`

→ display ^{along with} commit ^{no} modified file name

Note: git restore take place at latest commit.



→ To see changes in particular commit,
git show commit ID

git reset → Move head to diff. Commit
used to undo commits or go
back in history.

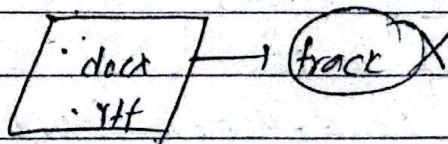
~~Similarities~~
~~git reset~~ → ~~Take me back to~~
~~Similarities~~ → ~~that commit but~~
git reset --soft keep all changes ready to commit again
git reset --mixed but unstage everything

Q7. Doing First Commit:

git add → add file to track
→ new file → add file to track in staging index
→ old file → staging index

similarities:
- moves head
- keeps changes in working directory

git Repository contains files:
- .cpp
- .js
- .css
- !



Difference:
git reset --soft
- keep changes staged
git reset --mixed
- unstaged changes

*** When we want some file not to be tracked i.e. (modification shouldn't be tracked)

Step 1: create a file which you don't want to be tracked.

Step 2: create a file named .gitignore (which is understood by git)

Step 3: put [*.extension of file] to ignore particular file in .gitignore

eg: ~~*.txt~~ *.txt (ignore all ~~edit~~ files with .txt pattern)
text file with any name)

* → means file of type extension of any pattern (i.e. filename)

eg: filename.fileextension eg: documentation.txt

eg: assets/images/*.png

because we have to put ~~filenam~~ correct files we need to ignore in git repository.

DATE

Page

Step 4: Add gitignore file to staging index.

git add .gitignore

steps: git commit -m "adding git ignore file".

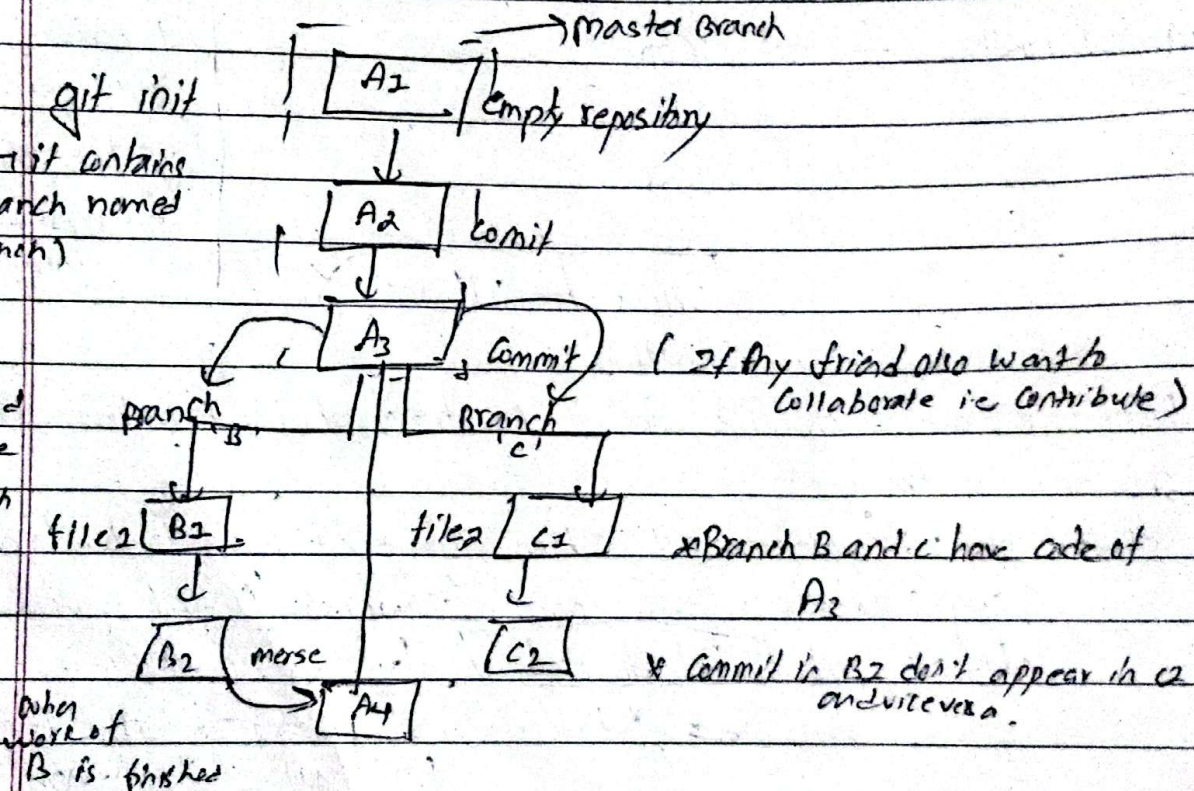
* When 2 or more person collaborate to do completely different works —
Branching, Tagging & Merging:

- git tag
- git branch
- git stash
- git checkout
- git merge

git init

(When init — it contains only one branch named master branch)

Branch 'B' and Branch 'C' are custom branch created.



When work of B is finished

Adv: of Branching and merging — master branch is supermaintained.

git branch branch-name ; to create a new branch
git checkout branch-name ; to switch to another branch

* How to create a branch: (Why → to create independent feature)

Command: git branch → list no. of branch and highlight current branch (with green)

git checkout -b branch-name { git branch quicksort → create a new branch
git checkout quicksort → switch from one branch to another (move)

When new branch is created and switch to that branch and a file is modified and committed and switch to another branch, then modified content will be removed

git checkout -b branch-name { branch-name is created and switched

git -am \$to perform add and commit

git add main.cpp.

git commit -am "bubblesort added"

* How to merge a multiple branch.

step 1: go to master branch (to merge in master branch)

git checkout master.

step 2: git merge branch-name (branch to merge)

* How to delete a branch.

git branch -d branch-name

*** Note: Merge conflict arise when merging master branch with another. ~~because~~ It is because on same line, different code arises, and git don't know which one to be taken place in that position.

cd -delete

git tag tagname commit-id -m "commit msg";
git tag -d tagname (to delete tag)

PRUEE

Date

Page

* How to resolve conflict:

git add file-name

git commit -m "msg"

* Tagging (need to tell it is ^{new} release)

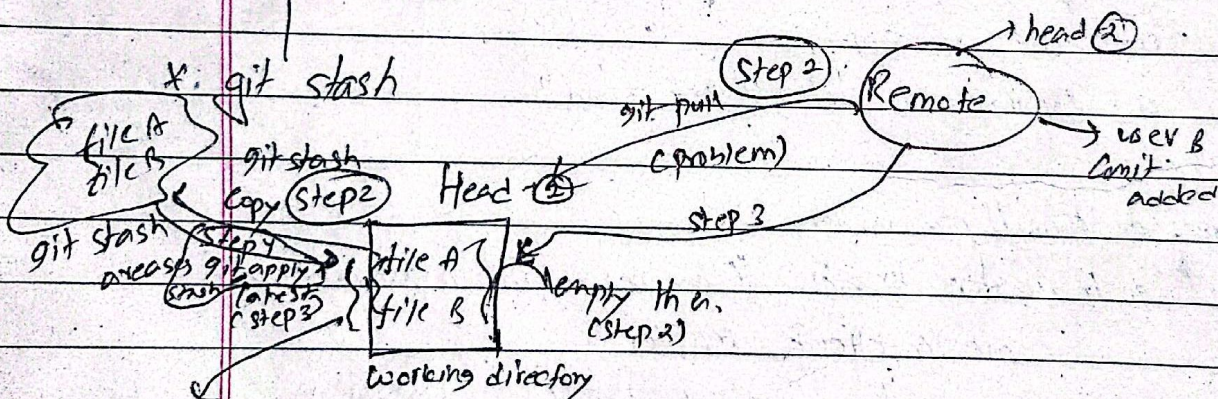
git tag -a beta v1.0 commit-id/latest -m "my beta release"

git commit -am "message" (shortcut for git add & git commit)
to remove tag:

git tag -d beta v1.0

Used when local has less no. of commit than center repo.

* git stash



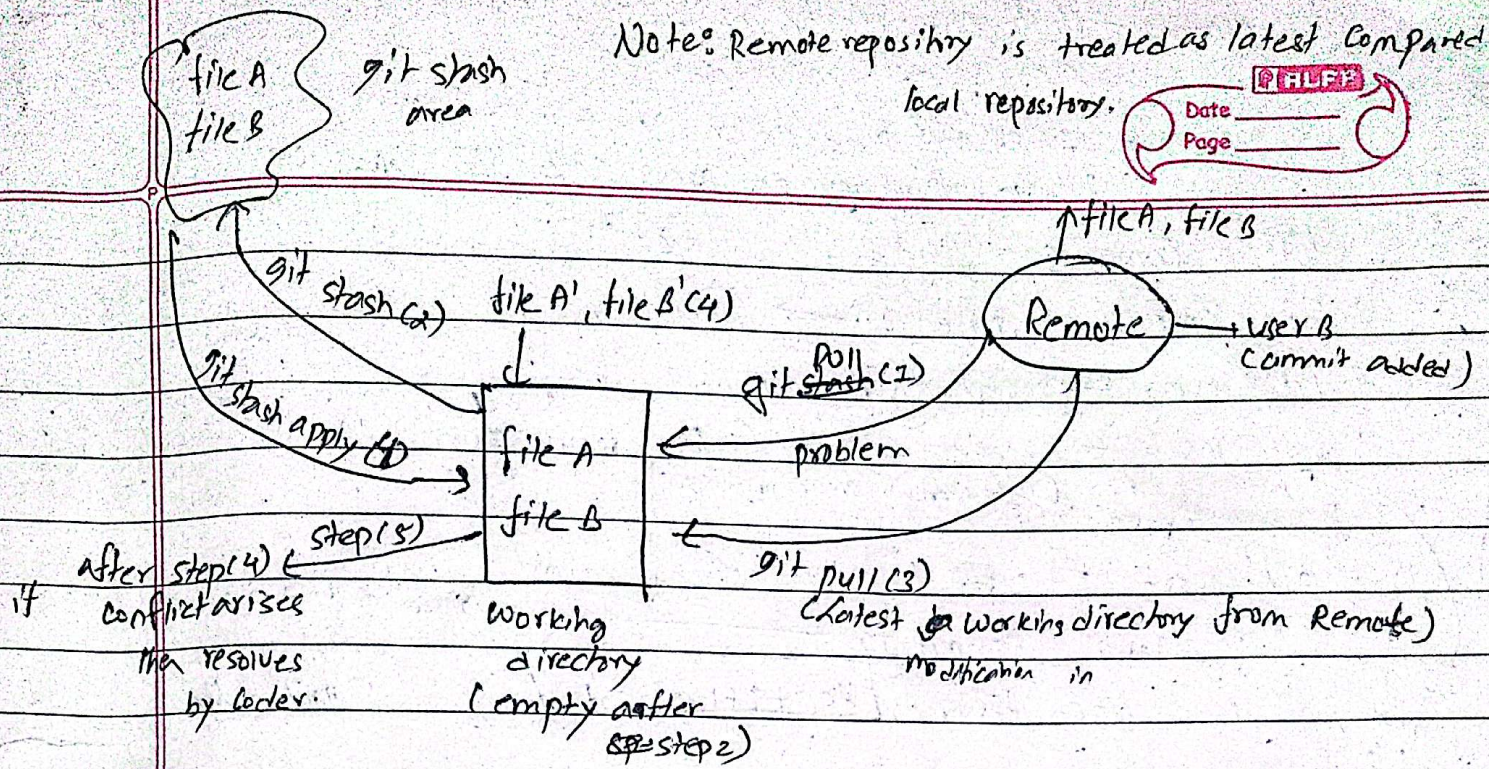
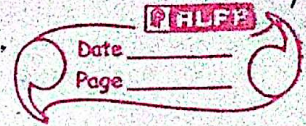
git pull → update repo. at local with repo. at remote.
Command: git pull (When git pull command is used on local as modify the, then move into a problem)

git stash

put ~~copy of~~ ^{from} file in working directory into areas (any area) and empty working directory.

Then file from remote is pull into local.

Note: Remote repository is treated as latest compared to local repository.



Note: After git-stash, working and staging index empty because ~~temp~~ temp changes move to stash.

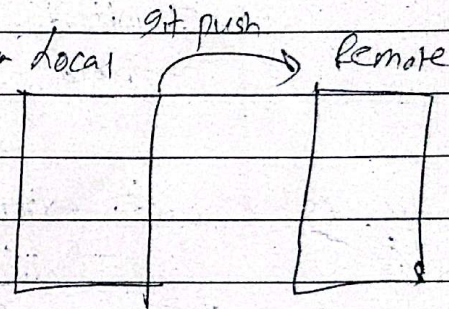
Step 2: git pull

Step 2: git stash

Step 3: git stash list # to view stash area local

Step 4: git pull

Step 5: git stash apply



git push -u origin master

Change Configuration setting

For own project - git config --global user.name ' '

git config --global user.email ' '

For client project - git config --local user.name ' '

git config --local user.email ' '

This is used to set author name & email to identity the author of commits.

changes made in all git repository will be

changes made in only one repository

Note: ~~local~~ both are same

to view configuration local or global
git config --local --list
git config --global --list

terminal → } to enter insert mode (edit message)

ESC to exit insert mod

type :wq and press enter. (w = write)

To exit without saving → press ESC, then type :q! and press enter

Date _____
Page _____

PROOF

Undo Commits:

• `git commit --amend` → amend the most recent commit i.e. (latest commit) message

• `git revert` → revert given commit

• `git reset` → delete commit (dangerous command)
i.e. move head to any commit id.

2) • `git revert` → used to revert wrong commit

`git revert commit-id`

to come out of terminal | ESC + : + wq + enter |

used to undo a commit without altering commit history.

• `git reset` → to remove unnecessary commit i.e.

`git reset --soft commit-id`

--mixed

--hard

Step 2: you make wrong commit i.e. modification is wrong

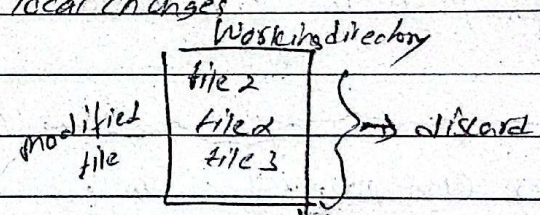
Step 2: you revert it to be correct
Commit 2
unnecessary commit has been created

Step 3: To remove it we use `git reset`

a) `git reset --hard` → remove unnecessary commit

→ discard changes i.e. files in working directory i.e. local changes

(avoid)
delete local changes



b) `git reset --mixed` → remove unnecessary commit

changes or files in
→ modification stages indexed to working directory

c) `git reset --soft` → ~~diff will show on stage as staged changes~~

→ remove unnecessary commit

→ do nothing with changes files.

i.e. - staging area & why exactly same
- working directory & why exactly same

is the place where head point now. all commit above it is gone.

used to move head to particular hash id.

git reset --soft

More detail in starting page.

(safe)

Local system

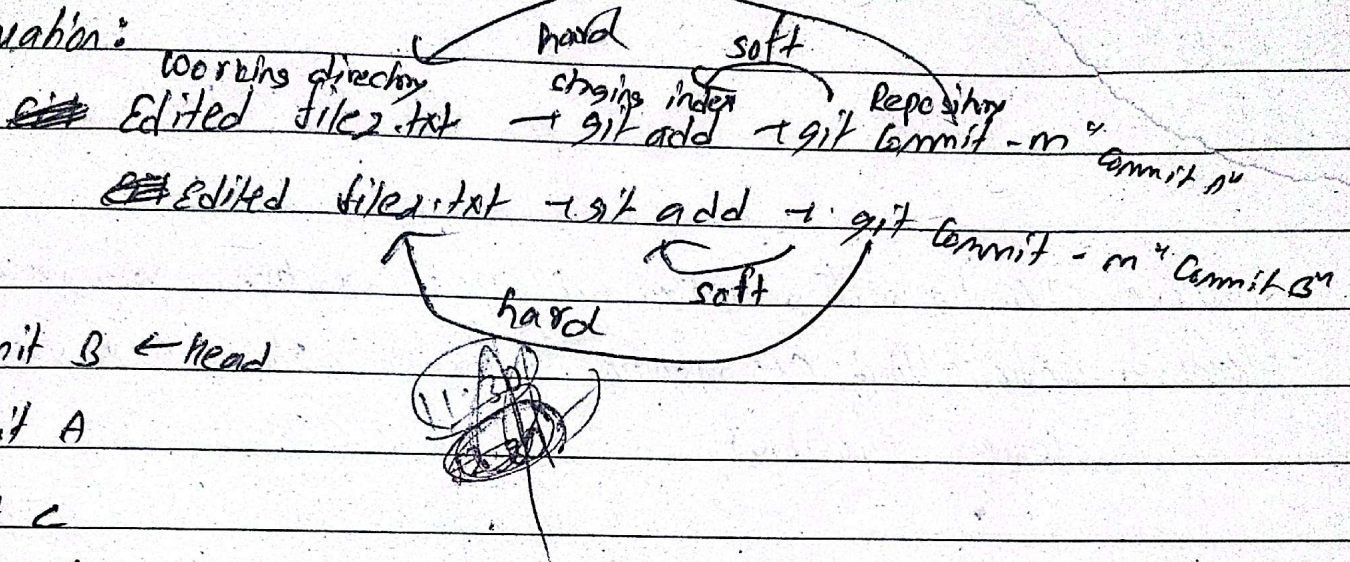
note system is **git**

Date _____

Page _____

`git reset --soft Commit.id / hash.id`

Situation:



Commit B ← head

Commit A

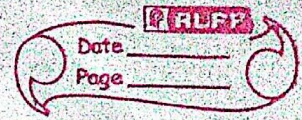
Commit C

If I do `git reset --soft Commit id of C`, then, `file2.txt` and `file1.txt` is about to commit (staging index)

If I do `git reset --mixed Commit id of C`, then `file2.txt` and `file1.txt` is modified (but not about to commit) i.e. changes are preserved but (in working directory)

If I do `git reset --hard Commit id of C` then changes made in `file2` & `file1.txt` will be discarded. The files.

git log --oneline → View summary of each commit i.e hash(ID) & Commit msg
 git show <commit-id> → to view details information of particular commit
 Used simultaneously if want to view details about commit.



* git commit --amend → to change description of latest commit i.e change commit message (at head or latest commit)

Note: amend will also change commit hash id.

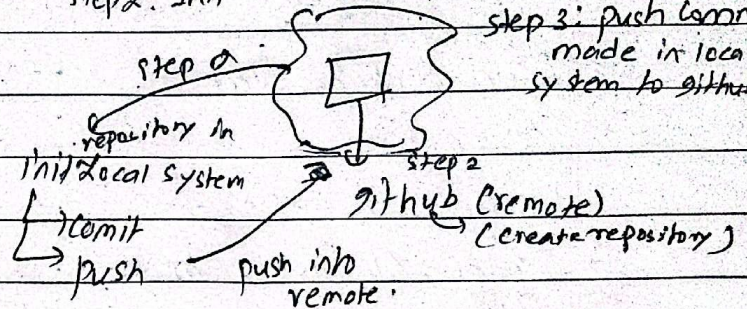
GIT GUIs & How to push ~~from~~ commit?

- Github Desktop
- sublime merge.

Step 1: Create repository in github

Step 2: Init it in local system

Step 3: push commit made in local system to github



* How to push commit?

Step 1: Create a git repository in system which can be pushed in remote

Step 2: create a new repository in github website (remote)

Step 3: create a repository in local system

Step 4: Create ~~Readme.md~~ file Readme.md file (Created as description text to tell about repository and its works)

Step 5: Commit Readme.md

Step 6: git remote add origin https://github.com/sujan2468/my-repo.git
 (Copy it from website after creating new repository)

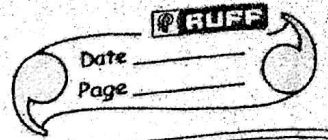
Step 7: (works: To map local branch with github remote branch)

Step 8: git push --set-upstream origin master (initially)

git push -u origin master (after)

git remote -v ⇒ Command is used to check local repo is linked with remote repo or not.

to check current git configuration,
git config --list



- global → Config settings apply to all git repositories on your system.
- local → Config settings are specific to single repository. (apply to the current repository)

✱ If you change latest commit message like
git commit --amend, then,

- git rewrite the commit history (gave it a new commit hash).
- your local branch master now has a diff commit than are on git hub.

When try to push → git push -u origin master

→ git sees that your local branch doesn't fast-forward from remote - meaning github has a commit that your local branch doesn't include (even though it's same content)

solution:

option 2: force push (safe if you're sure)

- if you're only one working on repo, you are okay overwriting the last commit.

→ git push -u origin master --force
or shorthand → git push -f